

✓ CSCI544 HW2: Sentiment Analysis with Word2Vec, Neural Networks, and CNNs

Name: Rudrani Chavarkar

USC ID: 3158210765

Library Used: Pytorch

```
%pip install gensim nltk torch
```

```
Requirement already satisfied: gensim in /usr/local/lib/python3.12/dist-packages (4.4.0)
Requirement already satisfied: nltk in /usr/local/lib/python3.12/dist-packages (3.9.1)
Requirement already satisfied: torch in /usr/local/lib/python3.12/dist-packages (2.9.0+cu128)
Requirement already satisfied: numpy>=1.18.5 in /usr/local/lib/python3.12/dist-packages (from gensim) (2.0.2)
Requirement already satisfied: scipy>=1.7.0 in /usr/local/lib/python3.12/dist-packages (from gensim) (1.16.3)
Requirement already satisfied: smart_open>=1.8.1 in /usr/local/lib/python3.12/dist-packages (from gensim) (7.5.0)
Requirement already satisfied: click in /usr/local/lib/python3.12/dist-packages (from nltk) (8.3.1)
Requirement already satisfied: joblib in /usr/local/lib/python3.12/dist-packages (from nltk) (1.5.3)
Requirement already satisfied: regex>=2021.8.3 in /usr/local/lib/python3.12/dist-packages (from nltk) (2025.11.3)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from nltk) (4.67.3)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from torch) (3.20.3)
Requirement already satisfied: typing-extensions>=4.10.0 in /usr/local/lib/python3.12/dist-packages (from torch) (4.15.0)
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from torch) (75.2.0)
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from torch) (1.14.0)
Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.12/dist-packages (from torch) (3.6.1)
Requirement already satisfied: Jinja2 in /usr/local/lib/python3.12/dist-packages (from torch) (3.1.6)
Requirement already satisfied: fsspec>=0.8.5 in /usr/local/lib/python3.12/dist-packages (from torch) (2025.3.0)
Requirement already satisfied: nvidia-cuda-nvrtc-cu12==12.8.93 in /usr/local/lib/python3.12/dist-packages (from torch) (12.8.93)
Requirement already satisfied: nvidia-cuda-runtime-cu12==12.8.90 in /usr/local/lib/python3.12/dist-packages (from torch) (12.8.90)
Requirement already satisfied: nvidia-cuda-cupti-cu12==12.8.90 in /usr/local/lib/python3.12/dist-packages (from torch) (12.8.90)
Requirement already satisfied: nvidia-cudnn-cu12==9.10.2.21 in /usr/local/lib/python3.12/dist-packages (from torch) (9.10.2.21)
Requirement already satisfied: nvidia-cublas-cu12==12.8.4.1 in /usr/local/lib/python3.12/dist-packages (from torch) (12.8.4.1)
Requirement already satisfied: nvidia-cufft-cu12==11.3.3.83 in /usr/local/lib/python3.12/dist-packages (from torch) (11.3.3.83)
Requirement already satisfied: nvidia-curand-cu12==10.3.9.90 in /usr/local/lib/python3.12/dist-packages (from torch) (10.3.9.90)
Requirement already satisfied: nvidia-cusolver-cu12==11.7.3.90 in /usr/local/lib/python3.12/dist-packages (from torch) (11.7.3.90)
Requirement already satisfied: nvidia-cusparselt-cu12==12.5.8.93 in /usr/local/lib/python3.12/dist-packages (from torch) (12.5.8.93)
Requirement already satisfied: nvidia-cusparse-cu12==12.5.8.93 in /usr/local/lib/python3.12/dist-packages (from torch) (12.5.8.93)
Requirement already satisfied: nvidia-cusparselt-cu12==0.7.1 in /usr/local/lib/python3.12/dist-packages (from torch) (0.7.1)
Requirement already satisfied: nvidia-nccl-cu12==2.27.5 in /usr/local/lib/python3.12/dist-packages (from torch) (2.27.5)
Requirement already satisfied: nvidia-nvshmem-cu12==3.3.20 in /usr/local/lib/python3.12/dist-packages (from torch) (3.3.20)
Requirement already satisfied: nvidia-nvtx-cu12==12.8.90 in /usr/local/lib/python3.12/dist-packages (from torch) (12.8.90)
Requirement already satisfied: nvidia-nvjitlink-cu12==12.8.93 in /usr/local/lib/python3.12/dist-packages (from torch) (12.8.93)
Requirement already satisfied: nvidia-cufile-cu12==1.13.1.3 in /usr/local/lib/python3.12/dist-packages (from torch) (1.13.1.3)
Requirement already satisfied: triton==3.5.0 in /usr/local/lib/python3.12/dist-packages (from torch) (3.5.0)
Requirement already satisfied: wrapt in /usr/local/lib/python3.12/dist-packages (from smart_open>=1.8.1->gensim) (2.1.1)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3->torch) (1.3.0)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from Jinja2->torch) (3.0.3)
```

```
# =====
# IMPORTS & CONFIGURATION
# =====

import numpy as np
import pandas as pd
import re
import os
import random
import json
import warnings
warnings.filterwarnings('ignore')

# Reproducibility
RANDOM_STATE = 42
random.seed(RANDOM_STATE)
np.random.seed(RANDOM_STATE)

# NLP
import nltk
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer
from nltk.tokenize import word_tokenize

# Word2Vec
import gensim
import gensim.downloader as api
from gensim.models import Word2Vec

# Sklearn
from sklearn.linear_model import Perceptron
from sklearn.svm import LinearSVC
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
from sklearn.preprocessing import LabelEncoder
```

```
# PyTorch
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader

torch.manual_seed(RANDOM_STATE)
if torch.cuda.is_available():
    torch.cuda.manual_seed(RANDOM_STATE)

DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f'Using device: {DEVICE}')
print(f'PyTorch version: {torch.__version__}')
print(f'Gensim version: {gensim.__version__}')

# Download NLTK resources
nltk.download('stopwords', quiet=True)
nltk.download('punkt', quiet=True)
nltk.download('punkt_tab', quiet=True)
```

```
Using device: cuda
PyTorch version: 2.9.0+cu128
Gensim version: 4.4.0
True
```

Question 1: Dataset Generation

```
# Load the data
!wget -nc https://web.archive.org/web/20201127142707if_/https://s3.amazonaws.com/amazon-reviews-pds/tsv/amazon_reviews_us_
--2026-02-15 03:36:10-- https://web.archive.org/web/20201127142707if_/https://s3.amazonaws.com/amazon-reviews-pds/tsv/amaz
Resolving web.archive.org (web.archive.org)... 207.241.237.3
Connecting to web.archive.org (web.archive.org)|207.241.237.3|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 512323500 (489M) [application/x-gzip]
Saving to: 'amazon_reviews_us_Office_Products_v1_00.tsv.gz'

amazon_reviews_us_0 100%[=====] 488.59M 17.7MB/s in 36s

2026-02-15 03:36:55 (13.7 MB/s) - 'amazon_reviews_us_Office_Products_v1_00.tsv.gz' saved [512323500/512323500]
```

```
RAW_DATA_PATH = 'amazon_reviews_us_Office_Products_v1_00.tsv.gz'
OUT_PATH = 'balanced_dataset_out.csv' # where the balanced dataset will be saved
N_PER_CLASS = 50000
RANDOM_STATE = 42

def load_df(path):
    if not os.path.exists(path):
        raise FileNotFoundError(f"{path} not found")
    # pandas will infer compression (.gz) automatically
    return pd.read_csv(path, sep='\t', on_bad_lines='skip', low_memory=False)

def standardize(df):
    # accept a few common column names
    if 'reviewText' in df.columns: df = df.rename(columns={'reviewText': 'text'})
    if 'review_body' in df.columns: df = df.rename(columns={'review_body': 'text'})
    if 'overall' in df.columns: df = df.rename(columns={'overall': 'rating'})
    if 'star_rating' in df.columns: df = df.rename(columns={'star_rating': 'rating'})
    if 'review' in df.columns and 'text' not in df.columns: df = df.rename(columns={'review': 'text'})

    if 'text' not in df.columns or 'rating' not in df.columns:
        raise ValueError("Data must contain 'text' and 'rating' columns (or common equivalents).")

    df = df[['text', 'rating']].dropna()
    df['rating'] = pd.to_numeric(df['rating'], errors='coerce').dropna().astype(int)
    return df[df['rating'].isin([1,2,3,4,5])]

def build_balanced(df, n_per_class=N_PER_CLASS, random_state=RANDOM_STATE):
    samples = []
    for r in range(1,6):
        grp = df[df['rating'] == r]
        if len(grp) == 0:
            continue
        samples.append(grp.sample(n=min(n_per_class, len(grp)), random_state=random_state))
    out = pd.concat(samples, ignore_index=True)
    out['label'] = out['rating'].apply(lambda x: 1 if x>3 else (2 if x<3 else 3))
    return out
```

```

if __name__ == '__main__':
    df_raw = load_df(RAW_DATA_PATH)
    df_std = standardize(df_raw)
    df_bal = build_balanced(df_std)
    df_bal.to_csv(OUT_PATH, index=False)
    print(f"Saved {len(df_bal)} rows to {OUT_PATH}")
    print("Label counts:\n", df_bal['label'].value_counts().sort_index())
    print("Rating counts:\n", df_bal['rating'].value_counts().sort_index())

```

Saved 250000 rows to balanced_dataset_out.csv

Label counts:

label

1 100000

2 100000

3 50000

Name: count, dtype: int64

Rating counts:

rating

1 50000

2 50000

3 50000

4 50000

5 50000

Name: count, dtype: int64

We constructed a balanced dataset of 250,000 Amazon reviews by randomly sampling 50,000 instances from each rating (1–5). This ensures that the dataset is not biased toward any particular star rating.

We then converted ratings into ternary sentiment labels:

- Ratings > 3 → Positive (Class 1)
- Ratings < 3 → Negative (Class 2)
- Rating = 3 → Neutral (Class 3)

This resulted in:

- 100,000 positive
- 100,000 negative
- 50,000 neutral

```

# ---- Train / Test Split (80% / 20%) ----
train_df, test_df = train_test_split(
    df_bal,
    test_size=0.2,
    random_state=RANDOM_STATE,
    stratify=df_bal['label']
)

print(f"Train size: {len(train_df)}")
print(f"Test size: {len(test_df)}")

print("\nTrain label distribution:")
print(train_df['label'].value_counts().sort_index())

print("\nTest label distribution:")
print(test_df['label'].value_counts().sort_index())

# ---- Binary Subset (Positive=1, Negative=2 only) ----
binary_train = train_df[train_df['label'].isin([1, 2]).reset_index(drop=True)]
binary_test = test_df[test_df['label'].isin([1, 2]).reset_index(drop=True)]

print(f"\nBinary Train size: {len(binary_train)}")
print(f"Binary Test size: {len(binary_test)}")

```

Train size: 200000

Test size: 50000

Train label distribution:

label

1 80000

2 80000

3 40000

Name: count, dtype: int64

Test label distribution:

label

1 20000

2 20000

3 10000

Name: count, dtype: int64

```
Binary Train size: 160000
Binary Test size: 40000
```

We used stratified 80/20 splitting to preserve label distribution in both training and testing sets.

Balancing is important because:

- It prevents majority-class bias.
- It ensures accuracy is a meaningful metric.
- It allows fair comparison across models.

✓ Text Preprocessing

```
# =====
# TEXT PREPROCESSING (same pipeline as HW1)
# =====
stop_words = set(stopwords.words('english'))
stemmer = PorterStemmer()

def preprocess_text(text):
    """
    Clean and preprocess a review:
    1. Lowercase
    2. Remove HTML tags
    3. Remove non-alphabetic characters
    4. Tokenize
    5. Remove stopwords
    6. Stem tokens
    Returns a list of tokens.
    """
    if not isinstance(text, str):
        return []
    text = text.lower()
    text = re.sub(r'<.*?>', '', text)          # Remove HTML
    text = re.sub(r'[^a-z\s]', '', text)       # Keep only letters/spaces
    tokens = word_tokenize(text)
    tokens = [t for t in tokens if t not in stop_words and len(t) > 1]
    tokens = [stemmer.stem(t) for t in tokens]
    return tokens

def preprocess_text_raw(text):
    """
    Similar to above but WITHOUT stemming (needed for Word2Vec lookup
    against pretrained vocab).
    """
    if not isinstance(text, str):
        return []
    text = text.lower()
    text = re.sub(r'<.*?>', '', text)
    text = re.sub(r'[^a-z\s]', '', text)
    tokens = word_tokenize(text)
    tokens = [t for t in tokens if t not in stop_words and len(t) > 1]
    return tokens

# ---- Tokenize all reviews (cached) ----
TOKENS_PATH = 'tokenized_reviews.npz'

if os.path.exists('tokens_stemmed.pkl') and os.path.exists('tokens_raw.pkl'):
    print('Loading cached tokens...')
    import pickle
    with open('tokens_stemmed.pkl', 'rb') as f:
        tokens_stemmed = pickle.load(f)
    with open('tokens_raw.pkl', 'rb') as f:
        tokens_raw = pickle.load(f)
else:
    print('Tokenizing all reviews (this may take a few minutes)...')
    import pickle
    tokens_stemmed = df_bal['text'].apply(preprocess_text).tolist()
    tokens_raw = df_bal['text'].apply(preprocess_text_raw).tolist()
    with open('tokens_stemmed.pkl', 'wb') as f:
        pickle.dump(tokens_stemmed, f)
    with open('tokens_raw.pkl', 'wb') as f:
        pickle.dump(tokens_raw, f)

df_bal['tokens_stemmed'] = tokens_stemmed
df_bal['tokens_raw'] = tokens_raw

# Re-attach tokens to train/test splits
```

```
train_df = df_bal.loc[train_df.index]
test_df = df_bal.loc[test_df.index]
binary_train = train_df[train_df['label'].isin([1, 2])].copy()
binary_test = test_df[test_df['label'].isin([1, 2])].copy()

print('Preprocessing complete.')
print('Sample tokenized review:', df_bal['tokens_raw'].iloc[0][:10])
```

Tokenizing all reviews (this may take a few minutes)...
Preprocessing complete.

Sample tokenized review: ['purchased', 'tabs', 'whim', 'put', 'movie', 'posters', 'used', 'four', 'inch', 'tabs']

Question 2: Word Embeddings

```
# =====
# QUESTION 2(a): Pretrained Word2Vec (word2vec-google-news-300)
# =====
print('Loading pretrained word2vec-google-news-300 model...')
print('(This will download ~1.6GB on first run)')

google_w2v = api.load('word2vec-google-news-300')
print('Pretrained model loaded. Vocab size:', len(google_w2v))

# ---- Semantic similarity examples ----
print('\n--- Semantic Similarity Examples (Pretrained) ---')

# Example 1: King - Man + Woman ≈ Queen
result1 = google_w2v.most_similar(positive=['king', 'woman'], negative=['man'], topn=5)
print('\nking - man + woman (top 5):')
for word, score in result1:
    print(f' {word}: {score:.4f}')

# Example 2: excellent ~ outstanding
sim2 = google_w2v.similarity('excellent', 'outstanding')
print(f'\nsimilarity(excellent, outstanding): {sim2:.4f}')

most_sim_excellent = google_w2v.most_similar('excellent', topn=5)
print('\nWords most similar to "excellent" (pretrained):')
for word, score in most_sim_excellent:
    print(f' {word}: {score:.4f}')
```

Loading pretrained word2vec-google-news-300 model...
(This will download ~1.6GB on first run)
[=====] 100.0% 1662.8/1662.8MB downloaded
Pretrained model loaded. Vocab size: 3000000

--- Semantic Similarity Examples (Pretrained) ---

king - man + woman (top 5):
queen: 0.7118
monarch: 0.6190
princess: 0.5902
crown_prince: 0.5499
prince: 0.5377

Similarity(excellent, outstanding): 0.5567

Words most similar to "excellent" (pretrained):
terrific: 0.7410
superb: 0.7063
exceptional: 0.6815
fantastic: 0.6803
good: 0.6443

```
# =====
# QUESTION 2(b): Train Custom Word2Vec on Amazon Reviews
# =====
CUSTOM_W2V_PATH = 'custom_w2v_300d.model'

if os.path.exists(CUSTOM_W2V_PATH):
    print('Loading cached custom Word2Vec model...')
    custom_w2v = Word2Vec.load(CUSTOM_W2V_PATH)
else:
    print('Training custom Word2Vec model (this may take several minutes)...')
    # Using raw tokens (no stemming) for Word2Vec training so that
    # vocabulary matches real English words and comparisons are meaningful.
    custom_w2v = Word2Vec(
        sentences=df_bal['tokens_raw'].tolist(),
        vector_size=300,      # embedding dimension
        window=11,           # context window
        min_count=10,         # ignore rare words
```

```

workers=4,
seed=RANDOM_STATE
)
custom_w2v.save(CUSTOM_W2V_PATH)
print('Custom Word2Vec model saved.')

print(f'Custom model vocab size: {len(custom_w2v.wv)}')

# ---- Semantic similarity examples (same as 2a) ----
print('\n--- Semantic Similarity Examples (Custom W2V) ---')

# Example 1: King - Man + Woman ≈ Queen
try:
    result1_custom = custom_w2v.wv.most_similar(
        positive=['king', 'woman'], negative=['man'], topn=5
    )
    print('\nking - man + woman (top 5):')
    for word, score in result1_custom:
        print(f' {word}: {score:.4f}')
except KeyError as e:
    print(f'Word not in custom vocab: {e}')

# Example 2: excellent ~ outstanding
try:
    sim2_custom = custom_w2v.wv.similarity('excellent', 'outstanding')
    print(f'\nSimilarity(excellent, outstanding): {sim2_custom:.4f}')

    most_sim_custom = custom_w2v.wv.most_similar('excellent', topn=5)
    print('\nWords most similar to "excellent" (custom):')
    for word, score in most_sim_custom:
        print(f' {word}: {score:.4f}')
except KeyError as e:
    print(f'Word not in custom vocab: {e}')

```

```

Training custom Word2Vec model (this may take several minutes)...
Custom Word2Vec model saved.
Custom model vocab size: 17564

```

```

--- Semantic Similarity Examples (Custom W2V) ---

```

```

king - man + woman (top 5):
kings: 0.5707
president: 0.5340
affairs: 0.4824
executive: 0.4783
peers: 0.4770

```

```

Similarity(excellent, outstanding): 0.7979

```

```

Words most similar to "excellent" (custom):
outstanding: 0.7979
superb: 0.7420
exceptional: 0.7392
terrific: 0.7094
fantastic: 0.6536

```

The pretrained Google News Word2Vec model is trained on approximately 100 billion words, while our custom model is trained only on 250K Amazon reviews.

Observations:

1. **General Semantic Analogies** The pretrained model correctly solves the analogy: king – man + woman ≈ queen This demonstrates strong global semantic structure.

The custom model fails to capture this analogy, likely because:

- The Amazon review corpus is small.
- The corpus does not frequently contain royal or political vocabulary.

2. **Domain-Specific Similarities** For words like "excellent" and "outstanding":

- The custom model shows higher similarity (0.7979 vs 0.5567).
- This suggests it captures sentiment-oriented relationships better.

Conclusion:

- Pretrained embeddings encode broader linguistic structure.
- Custom embeddings capture domain-specific sentiment nuances.
- For sentiment analysis, domain-specific embeddings may outperform general embeddings.

Feature Extraction Utilities

```
# =====
# FEATURE EXTRACTION UTILITIES
# =====
EMBED_DIM = 300
N_CONCAT = 10 # for Question 4(b)
MAX_LEN = 50 # for CNN (Question 5)

def get_avg_vector(tokens, model, is_gensim_kv=True):
    """
    Compute the mean Word2Vec vector for a list of tokens.
    Falls back to zero vector if no tokens are in the vocabulary.
    is_gensim_kv: True for KeyedVectors (pretrained), False for full Word2Vec model.
    """
    wv = model if is_gensim_kv else model.wv
    vecs = [wv[t] for t in tokens if t in wv]
    if not vecs:
        return np.zeros(EMBED_DIM)
    return np.mean(vecs, axis=0)

def get_concat_vector(tokens, model, n=10, is_gensim_kv=True):
    """
    Concatenate the first n Word2Vec vectors for a review.
    Pads with zeros if fewer than n tokens are in vocab.
    Returns a vector of size n * EMBED_DIM.
    """
    wv = model if is_gensim_kv else model.wv
    vecs = [wv[t] for t in tokens if t in wv]
    vecs = vecs[:n] # take first n
    while len(vecs) < n:
        vecs.append(np.zeros(EMBED_DIM))
    return np.concatenate(vecs) # shape: (n * EMBED_DIM,)

def get_padded_sequence(tokens, model, max_len=50, is_gensim_kv=True):
    """
    Return a padded/truncated sequence of Word2Vec vectors.
    Shape: (max_len, EMBED_DIM).
    """
    wv = model if is_gensim_kv else model.wv
    vecs = [wv[t] for t in tokens if t in wv]
    # Truncate
    vecs = vecs[:max_len]
    # Pad with zeros
    while len(vecs) < max_len:
        vecs.append(np.zeros(EMBED_DIM))
    return np.array(vecs) # shape: (max_len, EMBED_DIM)

def extract_features(df_subset, model, feature_type='avg', is_gensim_kv=True,
                    use_stemmed=False):
    """
    Extract features for an entire DataFrame subset.
    feature_type: 'avg' | 'concat' | 'padded'
    use_stemmed: if True, use stemmed tokens; else use raw tokens.
    """
    tok_col = 'tokens_stemmed' if use_stemmed else 'tokens_raw'
    features = []
    for tokens in df_subset[tok_col]:
        if feature_type == 'avg':
            feat = get_avg_vector(tokens, model, is_gensim_kv)
        elif feature_type == 'concat':
            feat = get_concat_vector(tokens, model, N_CONCAT, is_gensim_kv)
        elif feature_type == 'padded':
            feat = get_padded_sequence(tokens, model, MAX_LEN, is_gensim_kv)
        features.append(feat)
    return np.array(features)

print('Feature extraction utilities defined.')
```

Feature extraction utilities defined.

```
# =====
# PRE-COMPUTE AVERAGE FEATURES FOR ALL SPLITS
# (binary & ternary, both W2V models)
# =====
print('Extracting average Word2Vec features (this may take a few minutes)...')

# ----- Pretrained Google W2V -----
X_train_bin_g = extract_features(binary_train, google_w2v, 'avg', True)
```

```

X_test_bin_g = extract_features(binary_test, google_w2v, 'avg', True)
X_train_ter_g = extract_features(train_df, google_w2v, 'avg', True)
X_test_ter_g = extract_features(test_df, google_w2v, 'avg', True)

y_train_bin = binary_train['label'].values
y_test_bin = binary_test['label'].values
y_train_ter = train_df['label'].values
y_test_ter = test_df['label'].values

print('Pretrained features: done.')

# ----- Custom W2V -----
X_train_bin_c = extract_features(binary_train, custom_w2v, 'avg', False)
X_test_bin_c = extract_features(binary_test, custom_w2v, 'avg', False)
X_train_ter_c = extract_features(train_df, custom_w2v, 'avg', False)
X_test_ter_c = extract_features(test_df, custom_w2v, 'avg', False)

print('Custom features: done.')
print(f'Feature shape (binary, Google): {X_train_bin_g.shape}')

```

```

Extracting average Word2Vec features (this may take a few minutes)...
Pretrained features: done.
Custom features: done.
Feature shape (binary, Google): (160000, 300)

```

Question 3: Simple Models (Perceptron & SVM)

```

# =====
# QUESTION 3: Perceptron and SVM with W2V Features
# (binary classification: class 1 vs class 2)
# =====

results = {} # Collect all accuracy values here

print('='*60)
print('QUESTION 3: Simple Models – Binary Classification')
print('='*60)

# ---- Perceptron ----
print('\n--- Perceptron ---')

# With Google W2V features
perc_g = Perceptron(random_state=RANDOM_STATE, max_iter=1000)
perc_g.fit(X_train_bin_g, y_train_bin)
acc_perc_google = accuracy_score(y_test_bin, perc_g.predict(X_test_bin_g))
results['Perceptron_GoogleW2V_Binary'] = acc_perc_google
print(f'Perceptron | Google W2V | Binary Accuracy: {acc_perc_google:.4f}')

# With Custom W2V features
perc_c = Perceptron(random_state=RANDOM_STATE, max_iter=1000)
perc_c.fit(X_train_bin_c, y_train_bin)
acc_perc_custom = accuracy_score(y_test_bin, perc_c.predict(X_test_bin_c))
results['Perceptron_CustomW2V_Binary'] = acc_perc_custom
print(f'Perceptron | Custom W2V | Binary Accuracy: {acc_perc_custom:.4f}')

# ---- SVM ----
print('\n--- Linear SVM ---')

# With Google W2V features
svm_g = LinearSVC(random_state=RANDOM_STATE, max_iter=2000)
svm_g.fit(X_train_bin_g, y_train_bin)
acc_svm_google = accuracy_score(y_test_bin, svm_g.predict(X_test_bin_g))
results['SVM_GoogleW2V_Binary'] = acc_svm_google
print(f'SVM | Google W2V | Binary Accuracy: {acc_svm_google:.4f}')

# With Custom W2V features
svm_c = LinearSVC(random_state=RANDOM_STATE, max_iter=2000)
svm_c.fit(X_train_bin_c, y_train_bin)
acc_svm_custom = accuracy_score(y_test_bin, svm_c.predict(X_test_bin_c))
results['SVM_CustomW2V_Binary'] = acc_svm_custom
print(f'SVM | Custom W2V | Binary Accuracy: {acc_svm_custom:.4f}')

```

```

=====
QUESTION 3: Simple Models – Binary Classification
=====

```

```

--- Perceptron ---
Perceptron | Google W2V | Binary Accuracy: 0.7619
Perceptron | Custom W2V | Binary Accuracy: 0.7714

```



```

--- Linear SVM ---
SVM      | Google W2V | Binary Accuracy: 0.8217
SVM      | Custom W2V | Binary Accuracy: 0.8498

```

Results show that custom Word2Vec embeddings outperform pretrained embeddings in both Perceptron and SVM models.

Why?

1. The custom model is trained directly on Amazon reviews, so its embedding space reflects sentiment-heavy vocabulary.
2. Words like "refund", "cheap", "durable", and "recommend" are positioned in a way that better separates positive and negative classes.
3. Pretrained embeddings capture general language structure but may not emphasize product-review sentiment relationships.

Additionally:

- SVM outperforms Perceptron.
- This is expected because SVM maximizes margin and handles high-dimensional data better.

Conclusion: Domain-specific embeddings are more effective for sentiment classification when paired with linear classifiers.

✓ Question 4: Feedforward Neural Networks (FFNN / MLP)

```

# =====
# QUESTION 4: MLP Helper Classes and Training Loop
# =====

class SentimentDataset(Dataset):
    """PyTorch Dataset for fixed-size feature vectors."""
    def __init__(self, X, y):
        # Labels must start from 0 for CrossEntropyLoss
        self.X = torch.tensor(X, dtype=torch.float32)
        self.y = torch.tensor(y - 1, dtype=torch.long) # shift 1,2,3 -> 0,1,2

    def __len__(self):
        return len(self.y)

    def __getitem__(self, idx):
        return self.X[idx], self.y[idx]

class MLP(nn.Module):
    """
    Feedforward MLP with two hidden layers.
    Architecture: input_dim -> 50 -> 10 -> n_classes
    """
    def __init__(self, input_dim, n_classes):
        super(MLP, self).__init__()
        self.net = nn.Sequential(
            nn.Linear(input_dim, 50),
            nn.ReLU(),
            nn.Dropout(0.3),
            nn.Linear(50, 10),
            nn.ReLU(),
            nn.Dropout(0.3),
            nn.Linear(10, n_classes)
        )

    def forward(self, x):
        return self.net(x)

def train_mlp(X_train, y_train, X_test, y_test,
              input_dim, n_classes, epochs=10, batch_size=256, lr=1e-3):
    """
    Train an MLP and return test accuracy.
    """
    torch.manual_seed(RANDOM_STATE)

    train_loader = DataLoader(
        SentimentDataset(X_train, y_train),
        batch_size=batch_size, shuffle=True
    )
    test_loader = DataLoader(
        SentimentDataset(X_test, y_test),
        batch_size=batch_size, shuffle=False
    )

```

```

model = MLP(input_dim, n_classes).to(DEVICE)
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=lr)
scheduler = optim.lr_scheduler.StepLR(optimizer, step_size=5, gamma=0.5)

for epoch in range(1, epochs + 1):
    model.train()
    total_loss = 0
    for X_batch, y_batch in train_loader:
        X_batch, y_batch = X_batch.to(DEVICE), y_batch.to(DEVICE)
        optimizer.zero_grad()
        loss = criterion(model(X_batch), y_batch)
        loss.backward()
        optimizer.step()
        total_loss += loss.item()
    scheduler.step()

    if epoch % 5 == 0:
        print(f' Epoch {epoch}/{epochs} | Loss: {total_loss/len(train_loader):.4f}')

# Evaluate
model.eval()
all_preds, all_true = [], []
with torch.no_grad():
    for X_batch, y_batch in test_loader:
        X_batch = X_batch.to(DEVICE)
        preds = model(X_batch).argmax(dim=1).cpu().numpy()
        all_preds.extend(preds)
        all_true.extend(y_batch.numpy())

acc = accuracy_score(all_true, all_preds)
return acc

print('MLP training utilities defined.')

```

MLP training utilities defined.

```

# =====
# QUESTION 4(a): MLP with Average W2V Vectors
# =====
print('='*60)
print('QUESTION 4(a): MLP with Average W2V Features')
print('='*60)

EPOCHS_MLP = 10

# --- Binary (class 1 vs 2) ---
print('\n[Google W2V | Binary]')
acc = train_mlp(
    X_train_bin_g, y_train_bin, X_test_bin_g, y_test_bin,
    input_dim=EMBED_DIM, n_classes=2, epochs=EPOCHS_MLP
)
results['MLP_Avg_GoogleW2V_Binary'] = acc
print(f'MLP | Avg | Google W2V | Binary Accuracy: {acc:.4f}')

print('\n[Custom W2V | Binary]')
acc = train_mlp(
    X_train_bin_c, y_train_bin, X_test_bin_c, y_test_bin,
    input_dim=EMBED_DIM, n_classes=2, epochs=EPOCHS_MLP
)
results['MLP_Avg_CustomW2V_Binary'] = acc
print(f'MLP | Avg | Custom W2V | Binary Accuracy: {acc:.4f}')

# --- Ternary (class 1, 2, 3) ---
print('\n[Google W2V | Ternary]')
acc = train_mlp(
    X_train_ter_g, y_train_ter, X_test_ter_g, y_test_ter,
    input_dim=EMBED_DIM, n_classes=3, epochs=EPOCHS_MLP
)
results['MLP_Avg_GoogleW2V_Ternary'] = acc
print(f'MLP | Avg | Google W2V | Ternary Accuracy: {acc:.4f}')

print('\n[Custom W2V | Ternary]')
acc = train_mlp(
    X_train_ter_c, y_train_ter, X_test_ter_c, y_test_ter,
    input_dim=EMBED_DIM, n_classes=3, epochs=EPOCHS_MLP
)
results['MLP_Avg_CustomW2V_Ternary'] = acc
print(f'MLP | Avg | Custom W2V | Ternary Accuracy: {acc:.4f}')

```

=====

QUESTION 4(a): MLP with Average W2V Features

```
=====
[Google W2V | Binary]
  Epoch 5/10 | Loss: 0.3953
  Epoch 10/10 | Loss: 0.3796
MLP | Avg | Google W2V | Binary Accuracy: 0.8436

[Custom W2V | Binary]
  Epoch 5/10 | Loss: 0.3479
  Epoch 10/10 | Loss: 0.3343
MLP | Avg | Custom W2V | Binary Accuracy: 0.8671

[Google W2V | Ternary]
  Epoch 5/10 | Loss: 0.7865
  Epoch 10/10 | Loss: 0.7718
MLP | Avg | Google W2V | Ternary Accuracy: 0.6813

[Custom W2V | Ternary]
  Epoch 5/10 | Loss: 0.7426
  Epoch 10/10 | Loss: 0.7292
MLP | Avg | Custom W2V | Ternary Accuracy: 0.7040
```

The MLP significantly improves over linear models because:

- It introduces nonlinear decision boundaries.
- Hidden layers allow interaction between embedding dimensions.
- Dropout helps prevent overfitting.

Custom embeddings again outperform pretrained embeddings, reinforcing that domain-specific representation improves downstream classification.

Binary classification achieves higher accuracy than ternary classification because:

- Neutral reviews (rating 3) are inherently ambiguous.
- Neutral sentiment often overlaps lexically with positive and negative reviews.

```
# =====
# QUESTION 4(b): MLP with Concatenated First-10 W2V Vectors
# =====
print('='*60)
print('QUESTION 4(b): MLP with Concatenated (first 10) W2V Features')
print('='*60)

CONCAT_DIM = N_CONCAT * EMBED_DIM # 10 * 300 = 3000

print('Extracting concatenated features...')
# Google W2V
Xc_train_bin_g = extract_features(binary_train, google_w2v, 'concat', True)
Xc_test_bin_g = extract_features(binary_test, google_w2v, 'concat', True)
Xc_train_ter_g = extract_features(train_df, google_w2v, 'concat', True)
Xc_test_ter_g = extract_features(test_df, google_w2v, 'concat', True)

# Custom W2V
Xc_train_bin_c = extract_features(binary_train, custom_w2v, 'concat', False)
Xc_test_bin_c = extract_features(binary_test, custom_w2v, 'concat', False)
Xc_train_ter_c = extract_features(train_df, custom_w2v, 'concat', False)
Xc_test_ter_c = extract_features(test_df, custom_w2v, 'concat', False)
print(f'Concat feature shape: {Xc_train_bin_g.shape}')

# --- Binary ---
print('\n[Google W2V | Binary]')
acc = train_mlp(
    Xc_train_bin_g, y_train_bin, Xc_test_bin_g, y_test_bin,
    input_dim=CONCAT_DIM, n_classes=2, epochs=EPOCHS_MLP
)
results['MLP_Concat_GoogleW2V_Binary'] = acc
print(f'MLP | Concat | Google W2V | Binary Accuracy: {acc:.4f}')

print('\n[Custom W2V | Binary]')
acc = train_mlp(
    Xc_train_bin_c, y_train_bin, Xc_test_bin_c, y_test_bin,
    input_dim=CONCAT_DIM, n_classes=2, epochs=EPOCHS_MLP
)
results['MLP_Concat_CustomW2V_Binary'] = acc
print(f'MLP | Concat | Custom W2V | Binary Accuracy: {acc:.4f}')

# --- Ternary ---
print('\n[Google W2V | Ternary]')
acc = train_mlp(
    Xc_train_ter_g, y_train_ter, Xc_test_ter_g, y_test_ter,
    input_dim=CONCAT_DIM, n_classes=3, epochs=EPOCHS_MLP
)
```

```

results['MLP_Concat_GoogleW2V_Ternary'] = acc
print(f'MLP | Concat | Google W2V | Ternary Accuracy: {acc:.4f}')

print('\n[Custom W2V | Ternary]')
acc = train_mlp(
    Xc_train_ter_c, y_train_ter, Xc_test_ter_c, y_test_ter,
    input_dim=CONCAT_DIM, n_classes=3, epochs=EPOCHS_MLP
)
results['MLP_Concat_CustomW2V_Ternary'] = acc
print(f'MLP | Concat | Custom W2V | Ternary Accuracy: {acc:.4f}')

```

```

=====
QUESTION 4(b): MLP with Concatenated (first 10) W2V Features
=====
Extracting concatenated features...
Concat feature shape: (160000, 3000)

[Google W2V | Binary]
Epoch 5/10 | Loss: 0.4039
Epoch 10/10 | Loss: 0.3151
MLP | Concat | Google W2V | Binary Accuracy: 0.7815

[Custom W2V | Binary]
Epoch 5/10 | Loss: 0.4005
Epoch 10/10 | Loss: 0.3368
MLP | Concat | Custom W2V | Binary Accuracy: 0.8015

[Google W2V | Ternary]
Epoch 5/10 | Loss: 0.8012
Epoch 10/10 | Loss: 0.7318
MLP | Concat | Google W2V | Ternary Accuracy: 0.6322

[Custom W2V | Ternary]
Epoch 5/10 | Loss: 0.7911
Epoch 10/10 | Loss: 0.7398
MLP | Concat | Custom W2V | Ternary Accuracy: 0.6490

```

Concatenating the first 10 word vectors preserves limited positional information.

However, performance decreases compared to averaged embeddings.

Reasons:

1. The model sees only the first 10 words.
2. Important sentiment cues often appear later in reviews.
3. Concatenation increases input dimensionality (3000 vs 300), increasing overfitting risk.

This demonstrates that: For sentiment classification, global semantic distribution (averaging) is often more robust than limited positional encoding.

✓ Question 5: Convolutional Neural Networks

```

# =====
# QUESTION 5: CNN for Sentiment Classification
# =====

class SequenceDataset(Dataset):
    """Dataset for padded/truncated sequences (used by CNN)."""
    def __init__(self, X, y):
        # X shape: (N, MAX_LEN, EMBED_DIM)
        # CNN expects (N, C_in, L) = (N, EMBED_DIM, MAX_LEN)
        self.X = torch.tensor(X, dtype=torch.float32).permute(0, 2, 1)
        self.y = torch.tensor(y - 1, dtype=torch.long) # shift to 0-indexed

    def __len__(self):
        return len(self.y)

    def __getitem__(self, idx):
        return self.X[idx], self.y[idx]

class TextCNN(nn.Module):
    """
    Two-layer 1D CNN for text classification.
    Layer 1: conv(in=embed_dim, out=50, kernel=3) -> ReLU -> MaxPool
    Layer 2: conv(in=50, out=10, kernel=3) -> ReLU -> Global MaxPool
    FC:      10 -> n_classes
    """
    def __init__(self, embed_dim, n_classes, max_len=50):
        super(TextCNN, self).__init__()

```

```

self.conv1 = nn.Conv1d(in_channels=embed_dim, out_channels=50, kernel_size=3, padding=1)
self.conv2 = nn.Conv1d(in_channels=50, out_channels=10, kernel_size=3, padding=1)
self.relu = nn.ReLU()
self.pool = nn.MaxPool1d(kernel_size=2, stride=2)
self.drop = nn.Dropout(0.3)

# Compute flattened size after two conv+pool layers
L = max_len
L = L // 2 # after pool1
L = L // 2 # after pool2
self.fc = nn.Linear(10 * L, n_classes)
self._flat_size = 10 * L

def forward(self, x):
    # x: (batch, embed_dim, seq_len)
    x = self.pool(self.relu(self.conv1(x))) # -> (batch, 50, seq_len//2)
    x = self.pool(self.relu(self.conv2(x))) # -> (batch, 10, seq_len//4)
    x = self.drop(x)
    x = x.view(x.size(0), -1) # flatten
    return self.fc(x)

def train_cnn(X_train, y_train, X_test, y_test,
              n_classes, epochs=10, batch_size=256, lr=1e-3):
    """
    Train the TextCNN and return test accuracy.
    X_train/X_test shape: (N, MAX_LEN, EMBED_DIM)
    """
    torch.manual_seed(RANDOM_STATE)

    train_loader = DataLoader(
        SequenceDataset(X_train, y_train),
        batch_size=batch_size, shuffle=True
    )
    test_loader = DataLoader(
        SequenceDataset(X_test, y_test),
        batch_size=batch_size, shuffle=False
    )

    model = TextCNN(EMBED_DIM, n_classes, MAX_LEN).to(DEVICE)
    criterion = nn.CrossEntropyLoss()
    optimizer = optim.Adam(model.parameters(), lr=lr)
    scheduler = optim.lr_scheduler.StepLR(optimizer, step_size=5, gamma=0.5)

    for epoch in range(1, epochs + 1):
        model.train()
        total_loss = 0
        for X_batch, y_batch in train_loader:
            X_batch, y_batch = X_batch.to(DEVICE), y_batch.to(DEVICE)
            optimizer.zero_grad()
            loss = criterion(model(X_batch), y_batch)
            loss.backward()
            optimizer.step()
            total_loss += loss.item()
        scheduler.step()

        if epoch % 5 == 0:
            print(f' Epoch {epoch}/{epochs} | Loss: {total_loss/len(train_loader):.4f}')

    # Evaluate
    model.eval()
    all_preds, all_true = [], []
    with torch.no_grad():
        for X_batch, y_batch in test_loader:
            X_batch = X_batch.to(DEVICE)
            preds = model(X_batch).argmax(dim=1).cpu().numpy()
            all_preds.extend(preds)
            all_true.extend(y_batch.numpy())

    acc = accuracy_score(all_true, all_preds)
    return acc

print('CNN utilities defined.')

```

CNN utilities defined.

```

# ---- Extract padded sequence features ----
print('Extracting padded sequence features for CNN...')

# Google W2V
Xs_train_bin_g = extract_features(binary_train, google_w2v, 'padded', True)
Xs_test_bin_g = extract_features(binary_test, google_w2v, 'padded', True)

```

```
Xs_train_ter_g = extract_features(train_df, google_w2v, 'padded', True)
Xs_test_ter_g = extract_features(test_df, google_w2v, 'padded', True)

# Custom W2V
Xs_train_bin_c = extract_features(binary_train, custom_w2v, 'padded', False)
Xs_test_bin_c = extract_features(binary_test, custom_w2v, 'padded', False)
Xs_train_ter_c = extract_features(train_df, custom_w2v, 'padded', False)
Xs_test_ter_c = extract_features(test_df, custom_w2v, 'padded', False)

print(f'Sequence feature shape (binary, Google): {Xs_train_bin_g.shape}')
```

Extracting padded sequence features for CNN...
Sequence feature shape (binary, Google): (160000, 50, 300)

```
# =====
# QUESTION 5: CNN Training
# =====

print('='*60)
print('QUESTION 5: CNN Training')
print('='*60)

EPOCHS_CNN = 10

# --- Binary ---
print('\n[Google W2V | Binary]')
acc = train_cnn(
    Xs_train_bin_g, y_train_bin, Xs_test_bin_g, y_test_bin,
    n_classes=2, epochs=EPOCHS_CNN
)
results['CNN_GoogleW2V_Binary'] = acc
print(f'CNN | Google W2V | Binary Accuracy: {acc:.4f}')

print('\n[Custom W2V | Binary]')
acc = train_cnn(
    Xs_train_bin_c, y_train_bin, Xs_test_bin_c, y_test_bin,
    n_classes=2, epochs=EPOCHS_CNN
)
results['CNN_CustomW2V_Binary'] = acc
print(f'CNN | Custom W2V | Binary Accuracy: {acc:.4f}')

# --- Ternary ---
print('\n[Google W2V | Ternary]')
acc = train_cnn(
    Xs_train_ter_g, y_train_ter, Xs_test_ter_g, y_test_ter,
    n_classes=3, epochs=EPOCHS_CNN
)
results['CNN_GoogleW2V_Ternary'] = acc
print(f'CNN | Google W2V | Ternary Accuracy: {acc:.4f}')

print('\n[Custom W2V | Ternary]')
acc = train_cnn(
    Xs_train_ter_c, y_train_ter, Xs_test_ter_c, y_test_ter,
    n_classes=3, epochs=EPOCHS_CNN
)
results['CNN_CustomW2V_Ternary'] = acc
print(f'CNN | Custom W2V | Ternary Accuracy: {acc:.4f}')
```

```
=====
QUESTION 5: CNN Training
=====

[Google W2V | Binary]
Epoch 5/10 | Loss: 0.2765
Epoch 10/10 | Loss: 0.2240
CNN | Google W2V | Binary Accuracy: 0.8710

[Custom W2V | Binary]
Epoch 5/10 | Loss: 0.2862
Epoch 10/10 | Loss: 0.2399
CNN | Custom W2V | Binary Accuracy: 0.8761

[Google W2V | Ternary]
Epoch 5/10 | Loss: 0.6617
Epoch 10/10 | Loss: 0.6128
CNN | Google W2V | Ternary Accuracy: 0.7075

[Custom W2V | Ternary]
Epoch 5/10 | Loss: 0.6751
Epoch 10/10 | Loss: 0.6317
CNN | Custom W2V | Ternary Accuracy: 0.7116
```

The CNN achieves the highest performance among all models.

Why?

1. CNN preserves word order.
2. Convolutional filters detect local n-gram patterns such as:
 - "not good"
 - "highly recommend"
 - "very disappointed"
3. Max pooling extracts the most important features across the review.
4. Unlike averaging, CNN does not destroy positional structure.

Binary classification performs better than ternary classification due to:

- Clearer separation between positive and negative.
- Ambiguity of neutral sentiment.

Custom embeddings again slightly outperform pretrained embeddings, confirming that domain-adapted embeddings are beneficial.

Summary of All 16 Accuracy Values

```
# =====
# SUMMARY: All 16 Requested Accuracy Values
# =====
print('='*70)
print('SUMMARY OF ALL ACCURACY VALUES')
print('='*70)

# --- Group 1: Simple Models (4 values) ---
print('\n--- Simple Models (Binary Only: class 1 vs class 2) ---')
print(f'1. Perceptron | Google W2V | Binary : {results["Perceptron_GoogleW2V_Binary"]:.4f}')
print(f'2. Perceptron | Custom W2V | Binary : {results["Perceptron_CustomW2V_Binary"]:.4f}')
print(f'3. SVM | Google W2V | Binary : {results["SVM_GoogleW2V_Binary"]:.4f}')
print(f'4. SVM | Custom W2V | Binary : {results["SVM_CustomW2V_Binary"]:.4f}')

# --- Group 2: FFNN Avg (4 values: 2 W2V * 2 tasks) ---
print('\n--- MLP (Avg W2V) ---')
print(f'5. MLP-Avg | Google W2V | Binary : {results["MLP_Avg_GoogleW2V_Binary"]:.4f}')
print(f'6. MLP-Avg | Custom W2V | Binary : {results["MLP_Avg_CustomW2V_Binary"]:.4f}')
print(f'7. MLP-Avg | Google W2V | Ternary : {results["MLP_Avg_GoogleW2V_Ternary"]:.4f}')
print(f'8. MLP-Avg | Custom W2V | Ternary : {results["MLP_Avg_CustomW2V_Ternary"]:.4f}')

# --- Group 3: FFNN Concat (4 values) ---
print('\n--- MLP (Concat first-10 W2V) ---')
print(f'9. MLP-Cat | Google W2V | Binary : {results["MLP_Concat_GoogleW2V_Binary"]:.4f}')
print(f'10. MLP-Cat | Custom W2V | Binary : {results["MLP_Concat_CustomW2V_Binary"]:.4f}')
print(f'11. MLP-Cat | Google W2V | Ternary : {results["MLP_Concat_GoogleW2V_Ternary"]:.4f}')
print(f'12. MLP-Cat | Custom W2V | Ternary : {results["MLP_Concat_CustomW2V_Ternary"]:.4f}')

# --- Group 4: CNN (4 values) ---
print('\n--- CNN ---')
print(f'13. CNN | Google W2V | Binary : {results["CNN_GoogleW2V_Binary"]:.4f}')
print(f'14. CNN | Custom W2V | Binary : {results["CNN_CustomW2V_Binary"]:.4f}')
print(f'15. CNN | Google W2V | Ternary : {results["CNN_GoogleW2V_Ternary"]:.4f}')
print(f'16. CNN | Custom W2V | Ternary : {results["CNN_CustomW2V_Ternary"]:.4f}')

print('\n' + '='*70)
```

```
=====
SUMMARY OF ALL ACCURACY VALUES
=====

--- Simple Models (Binary Only: class 1 vs class 2) ---
1. Perceptron | Google W2V | Binary : 0.7619
2. Perceptron | Custom W2V | Binary : 0.7714
3. SVM | Google W2V | Binary : 0.8217
4. SVM | Custom W2V | Binary : 0.8498

--- MLP (Avg W2V) ---
5. MLP-Avg | Google W2V | Binary : 0.8436
6. MLP-Avg | Custom W2V | Binary : 0.8671
7. MLP-Avg | Google W2V | Ternary : 0.6813
8. MLP-Avg | Custom W2V | Ternary : 0.7040

--- MLP (Concat first-10 W2V) ---
9. MLP-Cat | Google W2V | Binary : 0.7815
10. MLP-Cat | Custom W2V | Binary : 0.8015
11. MLP-Cat | Google W2V | Ternary : 0.6322
12. MLP-Cat | Custom W2V | Ternary : 0.6490

--- CNN ---
```

13. CNN	Google W2V	Binary : 0.8710
14. CNN	Custom W2V	Binary : 0.8761
15. CNN	Google W2V	Ternary : 0.7075
16. CNN	Custom W2V	Ternary : 0.7116

=====

Overall Model Comparison

Model ranking (Binary Classification):

1. CNN (Custom W2V) – 0.8761
2. CNN (Google W2V)
3. MLP (Avg Custom W2V)
4. SVM (Custom W2V)
5. Perceptron

Key Insights:

- Increasing model complexity improves performance.
- Representation learning matters more than classifier complexity.
- Domain-specific embeddings consistently outperform general embeddings.
- CNN benefits from local context modeling.

Key Takeaways

1. Word embeddings dramatically outperform bag-of-words.
2. Domain adaptation improves classification.
3. Neural networks outperform linear models.
4. CNN is best for capturing phrase-level sentiment.
5. Neutral sentiment remains hardest to classify.

This experiment demonstrates the progression: Bag-of-Words → Word2Vec → MLP → CNN

Each step adds representational power.